ADR-004: how a YARG client reaches a remote library

Status: increment 1 **built and verified** 2026-09-07 (`yarg` `6a05002` on `dev`). Increments 2 and 3 remain proposals. **Constrained by ADR-001. Informed by UPSTREAM.md.**

Measured against `yarg` at `3673672` (branch `dev`) with the `YARG.Core` submodule at `028969a` . Every file and line cited below was read on 2026-09-07; where this document says "there is none", that is a grep with a count, not an impression.

Context

Phase 2 gave us a server that hands out ordinary `.sng` files and a `yarg-sync` client that mirrors a server's library into a local folder an **unmodified** YARG then plays. That works today and is the reason this project is useful to somebody now rather than after a merge.

It also has a ceiling: the player has to hold the whole library on disk, and has to run a separate tool to get it. The Phase 3 question is what the game itself should do — and, separately, what could plausibly be upstreamed.

`docs/UPSTREAM.md` covers the political half: their `CONTRIBUTING.md` sorts every feature into six tiers and the tier decides whether a PR is read at all, the tier for a remote song source is not published, and the answer is a Discord question that has not been asked yet. This ADR is the technical half, and it deliberately does not wait for that answer — the decision below is designed so their answer changes the *destination* of the work, not the work.

What the code actually says

Six findings. They are what the decision rests on, so they are stated with citations rather than summarised.

1. There is no networking in YARG.Core at all.

```
grep -rniE "HttpClient|System\.Net|UnityWebRequest|WebRequest" YARG.Core/ --include=*.cs
0
```

Zero matches. The Unity project *does* do networking — `Assets/Script/Integration/DiscordController.cs`, `Localization/LocalizationManager.cs`, `Song/SongSources.cs` — but none of it produces a `SongEntry`. So a remote source is not "extend the existing pattern". There is no pattern.

(`yarg/docs/song_sync.md` is about audio/clock synchronisation during play. It is not prior art for song delivery, and the name is a trap.)

2. The only public scan API takes a list of local directories.

`Song/Cache/CacheHandler.cs:43`

```
public static SongCache RunScan(bool tryQuickScan, string cacheLocation, string
badSongsLocation,
                                bool fullDirectoryPlaylists, List<string> baseDirectories)
```

That is the whole public surface of the scanner. `List<string>` of paths is baked in from `RunScan` down through `IniEntryGroup(dir)`, `ScanDirectory(DirectoryInfo, …)`, `ScanFile(FileInfo, …)`. There is no injectable source or provider interface to implement.

3. Exactly one seam over "where the bytes come from" already exists, and it is not enough.

`Song/Entries/Ini/SongEntry.IniBase.cs:82`

```
protected abstract FixedArray<byte>? GetChartData(string filename);
```

`LoadChart()` (`:92`) is built on it and is genuinely source-agnostic. **Everything else is not.** `SngEntry` calls `SngFile.TryLoadFromFile(_location, …)` at seven sites (`SongEntry.Sng.cs:33, 45, 73, 103, 199, 219, 241`) and touches `File.Exists` / `File.OpenRead` / `FixedArray.LoadFile` at nine more. `_location` (`SongEntry.IniBase.cs:71`) is a `string` set once in the constructor and is both `ActualLocation` and `SortBasedLocation`.

`SngFile` has exactly one loader — `TryLoadFromFile(string filename, bool loadMetadata)` (`IO/SngHandler/SngFile.cs:100`) — and it opens a `FileStream` directly. There is no stream-taking overload. `FixedArray` already has `ReadRemainder(Stream)` and `Read(Stream, long, bool)` (`IO/FixedArray/FixedArray.cs:28, 40`), so adding one is small, but it does not exist today.

4. Both ini entry classes are `internal sealed` with private constructors.

`SngEntry` (`SongEntry.Sng.cs:14`, ctor `:339`), `UnpackedIniEntry` (`SongEntry.UnpackedIni.cs:15`, ctor `:281`), and their base `IniSubEntry` (`SongEntry.IniBase.cs:37`) are all `internal`. A remote entry type **cannot** be added from

outside the assembly. Any entry-level approach is a change to YARG.Core itself, not an extension of it.

5. Scanning needs the chart bytes; deserialising does not.

`IniSubEntry.ScanChart` (`SongEntry.IniBase.cs:152`) is handed the full chart file, fills `AvailableParts` through the `Midi_*_Preparser` family (`SongEntry.Scanning.cs:9`), rejects with `ScanResult.NoNotes` unless `IsValid(in parts)` (`Scanning.cs:108`), and computes `entry._hash = HashWrapper.Hash(file.ReadOnlySpan)` (`IniBase.cs:214`). That hash is the key of `SongCache.Entries` (`Song/Cache/SongCache.cs:8`).

So an entry cannot be built from metadata alone **through the scan path**. It can through the cache path: `SngEntry.ForceDeserialize` (`SongEntry.Sng.cs:377`) reads hash, parts and metadata straight out of the cache and never touches the file, where its sibling `TryDeserialize` (`:355`) calls `AbridgedFileInfo.Validate` and `SngFile.ValidateMatch` first. **That asymmetry is the closest thing in the codebase to a precedent for a server-supplied entry.**

6. The cache is versioned and stores absolute paths.

`CacheHandler.cs:38` — `private const int CACHE_VERSION = 26_09_04_00;` (a `YY_MM_DD_RR` date-revision, checked at `:943`, written at `:965`; a mismatch discards the whole cache). `IniEntryGroup.Serialize` writes the group's absolute `_directory` (`IniEntryGroup.cs:38`) and then per-entry paths relative to it (`:53`).

This is independent confirmation of a permanent non-goal already on the books: **we do not generate `songcache.bin`.** It has a version constant that moves whenever upstream feels like it, and it embeds machine-specific absolute paths.

One more, because it is load-bearing for anything that serves `.sng`: **YARG does not validate the `.sng` version field.** `SngFile.TryLoadFromFile` reads it into `sng.Version` and moves on (`SngFile.cs:120–122`); the only use is `ValidateMatch(filename, versionToMatch)` (`:144`), which compares the file against the *cached* value to detect a changed file. Our `sng.Version1` comment already says this. It is now measured.

Decision

Build the remote source in the Unity layer, as three increments, and commit now only to the first.

The shape of that is set by finding 1 and finding 4 together: YARG.Core has no networking and cannot be extended from outside, and the Unity project already does networking. So **YARG.Core stays offline and the Unity layer fetches.** A design that puts an `HttpClient` inside YARG.Core is both the larger change and the harder one to upstream, and nothing requires it.

Increment 1 — the managed mirror folder. Zero YARG.Core changes.

The game keeps a folder it owns, mirrors a server's library into it exactly as `yarg-sync` does today, and adds that folder to the scan list. Everything downstream — scanning, caching, playing, sorting — is the path YARG already walks for a folder of `.sng` files.

Upstream already does exactly this, and that is the strongest argument in this document. `SongContainer.RunRefresh` (`Assets/Script/Song/SongContainer.cs:153`) builds its directory list from the player's `SongFolders` **and then appends one more**:

```
var directories = new List<string>(SettingsManager.Settings.SongFolders);
string setlistPath = PathHelper.SetlistPath;
if (!string.IsNullOrEmpty(setlistPath) && !directories.Contains(setlistPath))
{
    directories.Add(setlistPath);
}
```

`PathHelper.SetlistPath` is not a folder the player added. `FindLauncherPaths()` (`Assets/Script/Helpers/PathHelper.cs:150`) reads the **YARC Launcher's** `settings.json`, takes its `download_location`, and derives `<that>\Setlists` — a folder populated by a separate program, delivered out of band, appended to the scan list silently, and skipped cleanly when it does not exist.

That is structurally identical to what increment 1 needs. The pattern is not new, it is not ours, and upstream wrote it. What we would be adding is a second producer for the same shape of folder — which makes this a much smaller thing to ask about than "support remote libraries", and a much smaller thing to review.

What it costs the player: the whole library on disk, and a sync that runs before play.

Built, and measured against a real server

`yarg` `6a05002`. `PathHelper.ServerLibraryPath`, `SongServerSync.Sync(url, dest)`, a hidden `SongServerUrl` setting and a **Sync From Song Server** button in the Songs tab. Nothing in YARG.Core was touched, exactly as this ADR predicted.

`Assets/Editor/SongServerSyncSmokeTest.cs` runs a real sync over a real network from batchmode and exits non-zero on any failure. Against the vault2 deployment:

```
server=23 had=0 downloaded=23 failed=0 unmanaged=0 bytes=238215
PASS - 23 songs mirrored and verified, second run downloaded nothing
```

238,215 bytes is byte-for-byte what four concurrent `yarg-sync` clients pulled from the same server, so the C# client and the Go one agree exactly — a cross-check neither could give on its own. Every file's chart hash is verified against the name it was served under, using YARG.Core's own `SngFile` and `HashWrapper`; that is SHA-1 over the chart bytes, precisely what

the scanner computes, so a file that passes is a file the scanner will agree with. Idempotence is asserted rather than assumed, because it is the property that makes this safe to run on every launch.

It never deletes. Only `<40 hex>.sng` is ours; everything else is counted and left alone. There is no prune.

The constraint this ADR missed: Unity blocks plain HTTP

The first smoke test failed with `InvalidOperationException: Insecure connection not allowed`. Unity's `insecureHttpOption` defaults to `NotAllowed`, and **a song server on a LAN is plain HTTP — that is the normal case, not an edge case.** Nothing in the six findings above predicted this, because it is a Player Setting rather than an API.

Changed to `DevelopmentOnly` rather than `AlwaysAllowed`: that unblocks the editor and development builds and ships nothing weaker to players. **What a release build should do is a real open question** — relax the setting, require HTTPS from the server, or ask the player to opt in per host — and it is now a question for upstream rather than a default quietly changed in a fork.

Gaps, stated rather than discovered later

- A **300 Multiple Choices** response (one chart hash served by several packages) is recorded as a failure rather than resolved. Choosing is the client's job and it is not this increment's.
- `SongServerUrl` is a **hidden setting** edited in `settings.json`. There is no `AbstractSetting<string>` visual in this project except the IPv4 one, so a URL row means a new setting type *and* a new prefab. The working half landed first; the row can follow without changing anything under it.

Increment 2 — fetch a song when it is played.

This is the increment that needs YARG.Core, and finding 3 says exactly how much: two seams, not a redesign.

- `SngFile.TryLoadFromStream(Stream, bool loadMetadata)` beside the existing file loader. `FixedArray` already reads from streams; this is plumbing, and it is useful to upstream independently of anything remote.
- A way for an entry to **materialise its bytes before a load**. The honest version of this is not a new abstraction over `_location` — that would touch sixteen call sites in `SngEntry` alone — but a single hook the load methods call first, whose default is a no-op.

An entry in this mode is a real local file by the time anything reads it, so the cache, the hash and `ValidateMatch` all keep working unchanged. The player still holds every song they have *played*, not every song in the catalog.

Not committed to. It is written down here so that the seam is chosen deliberately when the time comes, rather than discovered under pressure.

Increment 3 — server-supplied entries, skipping the scan entirely.

A 30,000-song server would make the client scan 30,000 downloaded charts to learn what the server already knows. The server has the hash, the parts and the metadata; `ForceDeserialize` (finding 5) proves an entry can be built from exactly that.

Deferred, and the reason is finding 6. Consuming that path means either speaking the cache's serialisation format — versioned by a constant that moves without notice, with no stability contract — or adding a parallel constructor to YARG.Core. The first is the `songcache.bin` non-goal wearing a hat. The second is a real API addition and needs upstream to want it.

Our own scale measurements say when this starts to matter: indexing is linear at ~0.57 ms per song, so 30,000 songs is ~17 s of client-side scan. Annoying, not disqualifying. Increment 3 is a performance feature, and it should not be built before increment 2 makes it observable.

What this deliberately does not do

- **No `HttpClient` in YARG.Core.** Finding 1 makes that a change of character to the library, not an addition to it.
- **No streaming audio playback over HTTP.** `LoadAudio` wants random access; serving that well is a different project. Increment 2 downloads the `.sng` and plays it locally.
- **No `songcache.bin` generation.** Unchanged permanent non-goal, now with a version constant and an absolute path to point at.
- **No CON/mogg decryption**, which upstream independently lists as out of scope.

Consequences

- **Phase 3 can start now.** Increment 1 touches only the fork's Unity layer and needs no answer from anyone.
- **There is a precedent to point at.** The Official Setlist is already delivered out of band by a separate program into a folder YARG appends to its own scan list. A remote library is a second producer for that same shape, not a new concept.
- **The upstream ask gets smaller and more concrete.** It is no longer "support remote libraries"; it is "take `SngFile.TryLoadFromStream`, and consider a materialise hook on `IniSubEntry`". Both are small, both are useful outside this feature, and neither drags networking into YARG.Core. That is a much better thing to put in a Discord question than a paragraph about song servers.
- **Party mode (upstream #860) gets its second half.** The browse-and-search half is already deployed and answers a 10,000-song catalog in ~140 ms; queueing from a phone needs the game, which is what increment 1 first puts in place.

- **A player on increment 1 pays disk for simplicity.** That is the right first trade: the Pi-with-a-small-card case that increments 2 and 3 exist for is exactly the case we have never measured on real hardware.

Open questions, for upstream and for us

1. **Which tier does a remote song source fall into?** Unpublished, and it decides whether any of this is upstreamable at all. The Discord post is drafted in `docs/UPSTREAM.md` and is waiting on Jay.
2. **Would they take `SngFile.TryLoadFromStream` on its own?** It is a small, general improvement with no remote-library baggage. If the answer is yes, that is worth doing first regardless of what happens to the rest.
3. **Is anyone upstream already on this?** Not searched exhaustively. Doing so is cheaper than being told.
4. **What does the managed mirror folder do about songs the player deleted by hand?** `yarg-sync` already treats anything that is not `<40 hex>.sng` as the player's own and never touches it. The in-game version inherits that rule and should say so where a player can read it.
